# TCP part 1: sequence numbers and ACKs

*how TCP never loses your data*

Watch the 10-min video

**In one line:** TCP numbers every byte; the receiver answers with an ACK that names the next byte it expects; a lost segment is sent again after three duplicate ACKs or a timeout.

**WORDS YOU'LL MEET TODAY**

- **Packet**: a small piece of data on the internet
- **Byte**: a tiny unit of data, about one letter
- **Sequence number**: where that piece belongs
- **Duplicate ACK**: the same ACK again: "there's a gap"
- **Timeout**: "I waited too long"
- **Fast retransmit**: resend after 3 duplicate ACKs
- **Router**: a machine that forwards packets
- **Segment**: one piece of data TCP sends
- **ACK**: a reply that says "got it"
- **Cumulative ACK**: one ACK confirms all before it
- **Retransmit**: send the same piece again
- **Round trip**: data going there + the ACK back

## 1 The problem: the network makes no promises

You send a photo to a friend on WhatsApp. To travel, it is cut into small pieces called **packets**, and each one crosses the internet on its own. On the way, one packet is lost. One arrives twice. Two arrive in the wrong order. And yet, your friend sees a perfect photo. Who fixes this? A protocol called **TCP**, the Transmission Control Protocol.

- **Lost.** A busy **router**, the machine that forwards packets, can simply throw one away.
- **Reordered.** Packets can take different roads, so a later one can overtake an earlier one.
- **Duplicated.** Sometimes a packet is sent twice.

TCP runs on the sender and the receiver only. It can't fix the network; it works around these three problems.

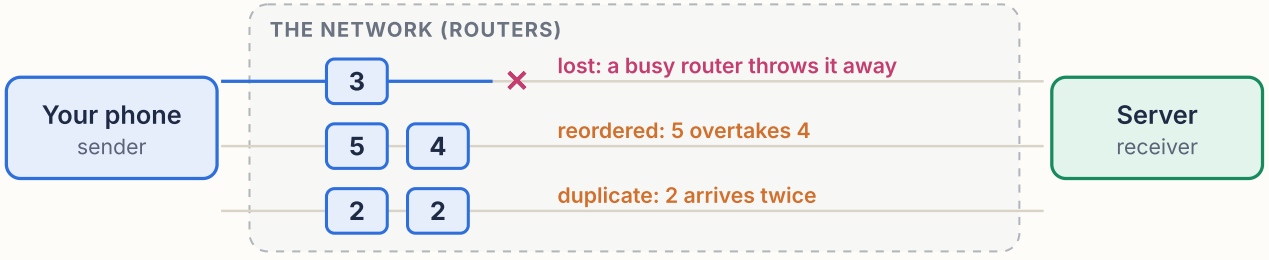



**Figure 1.** The network makes no promises. TCP runs only on the two ends and works around all three problems.

## 2 An everyday picture first: a letter by post

You post a five-page letter to a friend, each page in its own envelope. The post office usually works, but sometimes an envelope goes missing, and the envelopes arrive in a random order.

- **You numbered the pages**, so your friend sorts them. If page 2 arrives twice, they throw one copy away.
- **Page 4 never arrives.** Your friend sees the gap: 5 is here, 4 is not. So they send a postcard: *"I have pages 1 to 3. Send page 4 next."* That postcard is the idea of an ACK.
- **No postcard comes?** After a few days, you post the page again. Waiting too long is a **timeout**. Posting again is a **retransmit**.

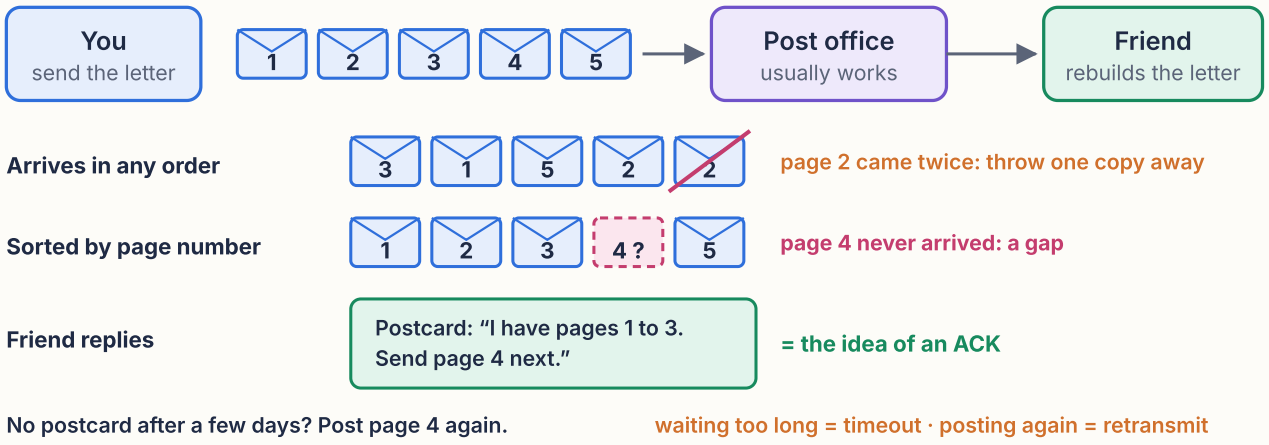



**Figure 2.** The everyday picture: page numbers to sort, a gap to ask about, silence to act on.

## 3 Now the real thing

| In the letter story | In TCP |
|---|---|
| You, posting the letter | the **sender**: your phone |
| Your friend | the **receiver**: the server |
| One envelope | a **segment**: your data plus a small TCP header, inside an IP packet |
| The page number | the **sequence number** |
| The postcard "send page 4 next" | the **ACK** |
| Waiting a few days / posting again | **timeout** / **retransmit** |

**One twist: TCP doesn't number pages, it numbers bytes.** A byte is a tiny unit of data, about one letter of text, and every byte gets its own number. Real numbers: your phone sends 3,000 bytes as three segments of 1,000 bytes. The first holds bytes 1 to 1,000, the second 1,001 to 2,000, the third 2,001 to 3,000. Each segment carries one number in its header, the number of its **first** byte, so the sequence numbers are **1, 1,001 and 2,001**.

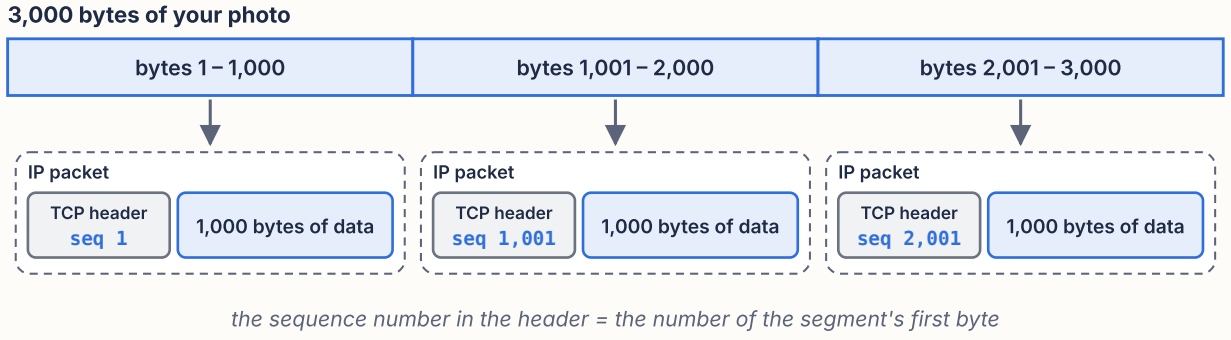



**Figure 3.** TCP numbers bytes, not pages: each segment carries the number of its first byte.

**One honest detail.** Real connections start counting from a random number (agreed in the handshake, lesson 9), but tools such as Wireshark show them from 1, and so do these notes.

## 4 The ACK: the most important idea today

ACK is short for **acknowledgement**: "I confirm I received it." The receiver sends ACKs back to the sender. Here is the rule, slowly:

```
ACK number = the next byte the receiver expects
```

The server receives bytes 1 to 1,000 and replies **ACK 1,001**. In plain words: *"I have everything up to 1,000. Send me 1,001 next."* Like a parcel tracking page, it shows one line: everything up to here has arrived. After the second segment the ACK becomes 2,001; after the third, 3,001. No gaps.

This is a **cumulative acknowledgement**: one ACK confirms everything before it. So if an ACK is lost on the way back, the next one covers it.

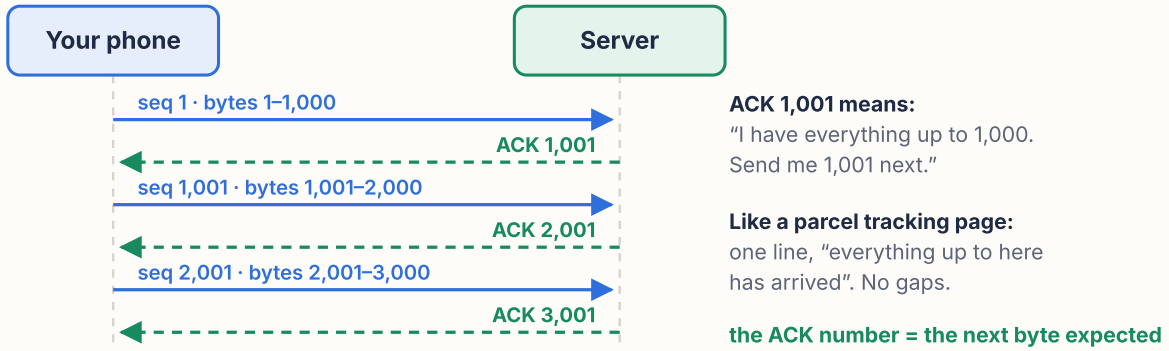



**Figure 4.** The ACK number is the next byte the receiver expects. One cumulative ACK confirms everything before it, so if one ACK is lost on the way back, the next one covers it.

## 5 Step by step: one segment is lost

The phone sends five segments, with sequence numbers 1, 1,001, 2,001, 3,001 and 4,001.

1. **Segment 1 arrives.** The server replies ACK 1,001. All good.
2. **Segment 1,001 is lost.** A busy router dropped it. Nothing arrives, so the server says nothing.
3. **Segment 2,001 arrives**, but bytes 1,001 to 2,000 are still missing. The server keeps the new segment in memory and repeats ACK 1,001: "I still need 1,001." An ACK that repeats the same number is a **duplicate ACK**. It means: there is a gap.
4. **Segment 3,001 arrives**, and the server sends ACK 1,001 again.
5. **Segment 4,001 arrives**, ACK 1,001 again. The phone now has **three duplicate ACKs**. It reasons: later segments get through, so one is missing. It resends segment 1,001 right away, without waiting. This is **fast retransmit**.
6. **The missing segment arrives.** The server now has every byte, 1 to 5,000, so the ACK jumps straight to **5,001**.

## 6 The real numbers, as a table

| Step | Phone sends | Arrives? | Server now holds | ACK sent back |
|---|---|---|---|---|
| 1 | seq 1 (bytes 1–1,000) | yes | 1–1,000 | **1,001** |
| 2 | seq 1,001 (1,001–2,000) | **lost** | 1–1,000 | nothing |
| 3 | seq 2,001 (2,001–3,000) | yes | 1–1,000, plus 2,001–3,000 kept aside | **1,001 (dup 1)** |
| 4 | seq 3,001 (3,001–4,000) | yes | 1–1,000, plus 2,001–4,000 kept aside | **1,001 (dup 2)** |
| 5 | seq 4,001 (4,001–5,000) | yes | 1–1,000, plus 2,001–5,000 kept aside | **1,001 (dup 3)** |
| | *three duplicate ACKs → fast retransmit of seq 1,001* | | | |
| 6 | seq 1,001 again | yes | 1–5,000, no gaps | **5,001** |

Read the last column: 1,001, then nothing, then 1,001 three times, then 5,001. The repeated number points straight at the gap.

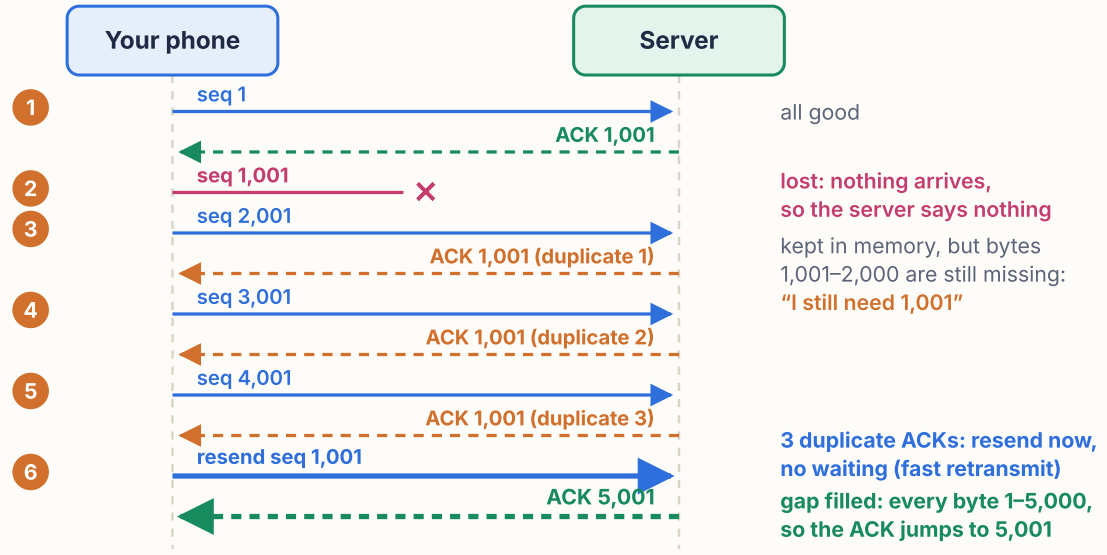



**Figure 5.** One lost segment: three duplicate ACK 1,001s, a fast retransmit, then ACK 5,001.

## 7 What goes wrong: the last segment is lost

If the lost segment was the last one, no later segments arrive, so there are no duplicate ACKs. Then the second safety net works: a **timer**. With every segment it sends, the sender starts a timer a little longer than one **round trip** (data going, and the ACK coming back). If it runs out with no ACK, that is a timeout: the sender retransmits, and **doubles the timer**, in case the network is struggling.

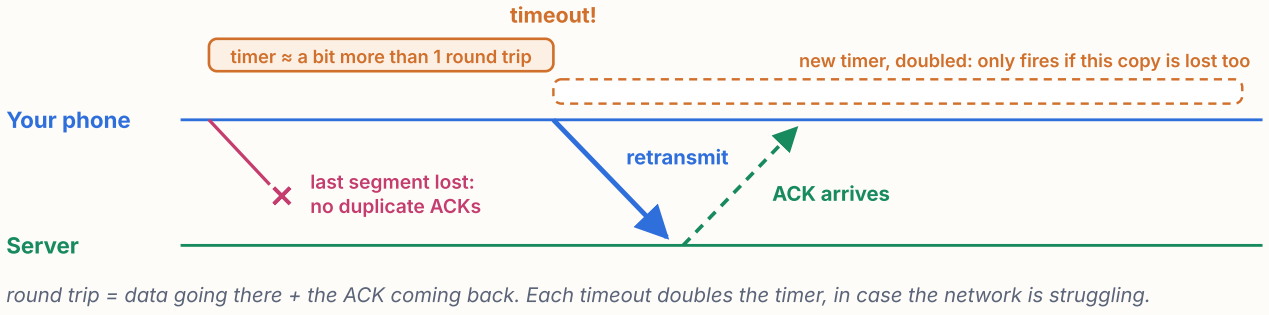



**Figure 6.** The second safety net. If no ACK comes before the timer runs out, the sender retransmits and doubles the timer.

**For the curious.** The timer is called the retransmission timeout (RTO). It is computed from measured round trips, with a floor (200 ms on Linux; the standard, RFC 6298, suggests 1 s). Real stacks run one timer for the oldest unacknowledged data rather than one per segment, and receivers often send one ACK for every two segments. None of this changes the logic: ACKs are cumulative.

PAUSE AND THINK

Four segments, starting at 1, 1,001, 2,001 and 3,001. The one at 2,001 is lost; the others arrive. Which ACK does the server keep sending? And after the resend arrives? Cover the answer and decide.

**Answer.** The server has bytes 1 to 2,000, so it keeps sending **ACK 2,001**. After the resend it has everything up to 4,000, so: **ACK 4,001**. If you got that, you understand ACKs.

Bonus: only one segment comes after the lost one, so there is only one duplicate ACK, not three. With the classic rule, this resend comes from the timer.

THREE COMMON MISTAKES

- **“Sequence numbers count segments.”** No, they count bytes. That's why they jump by 1,000.
- **“TCP guarantees fast delivery.”** No. It guarantees your data arrives complete and in order, or you get an error. Resending takes time, so on a weak mobile signal a page loads slowly, but nothing is lost.
- **“An ACK means the app handled the data.”** No. The receiver's operating system sends the ACK when the bytes reach its memory; the app may not have read them yet. That's why apps add their own confirmations, like WhatsApp's ticks, or a UPI success screen.

RECAP IN THREE LINES

1. TCP numbers every byte, and each segment carries the number of its first byte.
2. The receiver ACKs the next byte it expects.
3. A lost segment is resent after three duplicate ACKs, or a timeout.

## 8 Interview questions: what to say in 30 seconds

**Q1. How does TCP make sure data is never lost?** BASIC

TCP numbers every byte. The receiver sends cumulative ACKs with the next byte it expects, and uses sequence numbers to reorder data and drop duplicates. A lost segment is retransmitted after three duplicate ACKs, called fast retransmit, or when the retransmission timer expires.

**Q2. The server sends “ACK 1,001”. What exactly does that mean?** BASIC

“I have every byte up to 1,000; send me 1,001 next.” It is the next byte expected, not the last one received, and it is cumulative: it confirms everything before it, so a lost ACK is covered by the next.

**Q3. Five 1,000-byte segments, the second is lost. Walk through the ACKs.** SCENARIO

ACK 1,001; then nothing for the lost segment; then ACK 1,001 three more times as 2,001, 3,001 and 4,001 arrive and are kept aside. Three duplicates trigger fast retransmit of 1,001; the server then has bytes 1 to 5,000 and answers ACK 5,001.

**Q4. What if the lost segment is the last one?** SCENARIO

No later segments, so no duplicate ACKs. The retransmission timer covers it: a little longer than one measured round trip. When it runs out with no ACK, the sender retransmits and doubles the timer (exponential backoff).

**Q5. Why wait for three duplicate ACKs, not one?** TRICKY

Reordering also causes a duplicate ACK or two with nothing lost. Three duplicates are strong evidence of a real loss while later data still gets through, so the sender resends without waiting for the slower timeout.

**Q6. The server's TCP ACKed your request. Has the app processed it?** TRICKY

Not necessarily. The receiver's operating system sends the ACK when the bytes reach its memory; the app may not have read them yet, or may crash first. That's why apps add their own confirmations, like WhatsApp's ticks or a UPI success screen.

**Read more.** RFC 9293 (TCP), RFC 5681 (fast retransmit, duplicate ACKs), RFC 6298 (the retransmission timer). The full written lesson is at backendroadmap.com.

Next: SD-N-009 · TCP part 2: the three-way handshake backendroadmap.com